Introducción a GPU y CUDA

Diego Feroldi
Nicolás Becerra

Arquitectura del Computador*
Departamento de Ciencias de la Computación
FCEIA-UNR



*Actualizado 30 de julio de 2026 (D. Feroldi, feroldi@fceia.unr.edu.ar)

Índice

1. Introducción **1**

2. Evolución de las GPU modernas **3**

3. CUDA **4**
3.1. Kernels en CUDA 5
3.2. Arquitectura de la GPU 6
3.2.1. Núcleos de la GPU: la unidad de cómputo más pequeña 7
3.2.2. Warps: grupos de hilos en la GPU 7
3.3. Hola mundo en CUDA 8
3.4. Otro ejemplo más complejo 10

4. Compilación de programas CUDA **11**
4.1. Compilación local 11
4.1.1. Instalación en Ubuntu 11
4.1.2. Configuración de variables de entorno 12
4.2. Compilacion Web 12

5. Bibliografía **12**

1. Introducción

Una **GPU (Graphics Processing Unit)** es un coprocesador especializado en ejecutar eficientemente un gran número de operaciones en paralelo, originalmente diseñado para el renderizado de gráficos por computadora. En comparación con una CPU, una GPU puede ofrecer una mayor tasa de procesamiento y un mayor ancho de banda de memoria. Esto se debe a que las GPU incorporan un gran número de unidades de procesamiento relativamente simples, capaces de ejecutar muchas operaciones de forma simultánea, lo que permite procesar grandes volúmenes de datos en paralelo.

Además, su subsistema de memoria está optimizado para transferir grandes cantidades de datos a alta velocidad, lo que favorece aplicaciones con alto grado de paralelismo, como el procesamiento gráfico o ciertos tipos de cómputos científicos. En consecuencia, su arquitectura está orientada a maximizar el rendimiento en tareas altamente paralelizables, dedicando más recursos al procesamiento simultáneo de datos que a la reducción de la latencia[1] individual de cada operación.

La diferencia entre CPU y GPU se debe principalmente a que fueron diseñadas con propósitos distintos. La CPU está optimizada para ejecutar secuencias de instrucciones de forma eficiente y con baja latencia. Para ello, dispone de un número reducido de unidades de ejecución altamente sofisticadas, capaces de explotar paralelismo a nivel de instrucciones y, en menor medida, paralelismo a nivel de hilos (*threads*)[2].

En contraste, la GPU está diseñada para maximizar el paralelismo masivo, permitiendo la ejecución concurrente de una gran cantidad de threads que realizan operaciones similares sobre distintos datos.

Las unidades de procesamiento gráfico (GPU) surgieron con un propósito específico: renderizar píxeles y polígonos a alta velocidad para la industria del entretenimiento. Sin embargo, su capacidad inherente para procesar grandes volúmenes de datos en paralelo impulsó una transformación profunda en su rol. En la actualidad, las GPU no solo dominan el ámbito de los videojuegos, sino que se han convertido en un componente fundamental del cálculo científico intensivo en diversas áreas, entre ellas la inteligencia artificial.

Para lograr este alto nivel de paralelismo, las GPU sacrifican algunos recursos presentes en las CPU, como grandes memorias caché y mecanismos complejos de control de flujo. Como consecuencia, ciertas operaciones pueden presentar mayor latencia o menor eficiencia en presencia de divergencia de control, aunque esto se compensa con la gran cantidad de threads que pueden ejecutarse simultáneamente.

[1] La latencia es el tiempo que transcurre entre el inicio de una operación y la obtención de su resultado.

[2] Un ***thread*** (o hilo de ejecución) es la unidad mínima de ejecución que puede ser planificada por el sistema operativo dentro de un proceso. Los threads pertenecientes a un mismo proceso comparten el mismo espacio de direcciones y recursos, pero pueden ejecutarse de manera concurrente (y, en sistemas con múltiples núcleos, también en paralelo).

En general, una aplicación combina tareas paralelas y secuenciales. Por esta razón, los sistemas actuales suelen distribuir el trabajo entre la CPU y la GPU. Las aplicaciones con un alto nivel de paralelismo pueden aprovechar la arquitectura de la GPU para alcanzar una eficiencia significativamente mayor que la que se obtendría utilizando únicamente la CPU.

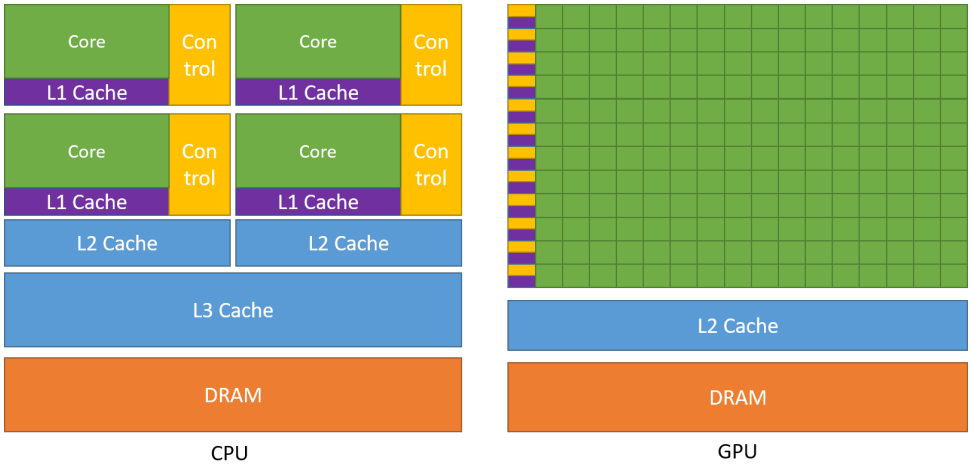


Figura 1: CPU (diseño orientado a reducir la latencia entre instrucciones) vs. GPU (diseño orientado a ejecutar una gran cantidad de instrucciones en paralelo).

La Fig. 1 compara de forma esquemática la organización interna de una CPU (izquierda) y una GPU (derecha). En la CPU se observan pocos núcleos de procesamiento (*cores*), cada uno con su propia memoria caché L1 y lógica de control compleja, además de niveles de memoria compartida L2 y L3 antes de acceder a la memoria principal (DRAM).

En contraste, la GPU presenta una gran cantidad de unidades de procesamiento más simples organizadas en paralelo, con relativamente menos lógica de control por unidad. Estas unidades se agrupan en multiprocesadores que disponen de memoria local (incluyendo caché L1 y memoria compartida), y comparten una memoria caché L2 antes de acceder a la DRAM global.

Esta organización refleja una diferencia fundamental de diseño: mientras que la CPU está optimizada para ejecutar un número reducido de tareas con baja latencia, la GPU prioriza el procesamiento masivo de hilos en paralelo, maximizando el *throughput* incluso a costa de una mayor latencia individual.

Cuando las GPU comenzaron a ser programables, algunos investigadores observaron que ciertos algoritmos originalmente no gráficos podían implementarse utilizando el lenguaje nativo de una GPU. En estos casos, la imagen resultante de aplicar el algoritmo contenía la solución al problema.

Dado que las GPU disponían de cientos de unidades de procesamiento (aunque a una frecuencia menor que la de una CPU), los algoritmos paralelos implementados de esta manera resultaban muchas veces más rápidos que sus implementaciones en CPU.

La arquitectura altamente paralela de las GPU plantea un desafío fundamental desde el punto de vista de la programación: ¿cómo expresar un problema de manera que pueda ser ejecutado eficientemente por una gran cantidad de unidades de procesamiento en paralelo?

Para abordar este problema, es necesario adoptar un modelo de programación que permita descomponer el trabajo en múltiples tareas independientes que puedan ejecutarse simultáneamente. En este contexto, surge el concepto de ***kernel***, una función que es ejecutada de manera concurrente por un gran número de threads sobre distintos datos.

Este enfoque da lugar a un modelo de ejecución específico, en el cual miles de threads cooperan para resolver un problema de forma paralela. Comprender cómo se organizan y ejecutan estos threads resulta fundamental para aprovechar el potencial de las GPU.

En la siguiente sección se introduce el modelo de ejecución de **CUDA**, que define cómo se estructuran y coordinan estos threads durante la ejecución de un programa.

2. Evolución de las GPU modernas

Una GPU moderna es un *coprocesador altamente paralelo* presente en la mayoría de las computadoras de uso personal. Los primeros ejemplos de este tipo de coprocesadores se remontan a computadoras de las décadas de 1970 y 1980, y podían tener distintos usos (dibujar gráficos en pantalla, alojar la memoria de video, etc.). Sin embargo, a partir de la década de 1990 las GPU comenzaron a converger hacia una misma función: acelerar la ejecución de algoritmos gráficos conocidos como algoritmos de *rasterización*.

Estos algoritmos toman como entrada formas geométricas en 3D (generalmente triángulos). Luego se utilizan matrices para trasladar, rotar y proyectar estos triángulos sobre la superficie plana del monitor, transformando coordenadas 3D en coordenadas 2D, ya que la pantalla es un plano. El paso clave de la rasterización consiste en determinar qué color asignar a cada píxel de la pantalla.

El procesamiento de cada forma geométrica se realizaba de manera independiente, siguiendo una arquitectura de *pipeline*. Las etapas de este pipeline eran originalmente fijas, aunque algunas podían parametrizarse para elegir, por ejemplo, colores, matrices de proyección o tamaños. Dado que cada forma geométrica se procesaba independientemente de las demás, surgió una arquitectura paralela con muchos procesadores dedicados a las mismas tareas, capaces de procesar múltiples formas simultáneamente. La Fig. 2 muestra un pipeline sencillo, cuyas etapas principales son:

Vertex Shader: procesa los vértices de la geometría.

Geometry Shader: puede generar o modificar la geometría.

Clipping: elimina las partes que quedan fuera del volumen visible.

Screen Mapping: transforma las coordenadas al espacio de pantalla.

Triangle Setup: prepara los triángulos para la rasterización.

Triangle Traversal: recorre los píxeles cubiertos por cada triángulo.

Pixel Shader: calcula el color de cada píxel.

Merger: combina los resultados de la etapa *pixel shader* con el contenido actual de la imagen.

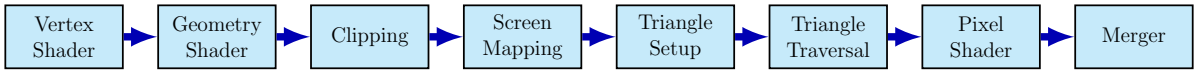


Figura 2: Pipeline simple para una GPU.

Con el tiempo se incorporaron más parámetros a las etapas fijas, pero pronto se hizo evidente que esta arquitectura resultaba limitada. Por esta razón, algunas etapas —especialmente *vertex shading* y *pixel* o *fragment shading*— se convirtieron en etapas programables. Para ello se definieron lenguajes específicos, conocidos como lenguajes de *shaders*, que permiten especificar, dentro de ciertas restricciones, el comportamiento de estas etapas.

Esta evolución implicó que los procesadores de las GPU se volvieran cada vez más generales. A mediados de la década de 2000, las GPU comenzaron a construirse con cientos de unidades de procesamiento capaces de ejecutar programas relativamente generales, lo que abrió la puerta al uso de las GPU para cómputo de propósito general (*GPGPU*).

Sin embargo, programar estas arquitecturas utilizando lenguajes de shaders resultaba complejo y poco natural para problemas no gráficos. Como respuesta a esta limitación, surgieron modelos y herramientas específicamente diseñados para cómputo general en GPU.

Entre ellos, NVIDIA desarrolló CUDA, un entorno de programación que permite escribir programas en un lenguaje de alto nivel y ejecutarlos directamente sobre la GPU, sin necesidad de reformular el problema en términos gráficos.

3. CUDA

CUDA (*Compute Unified Device Architecture*) es una arquitectura de computación paralela desarrollada por NVIDIA que permite utilizar las GPU para cómputo de propósito general (comúnmente denominado GPGPU, *General-Purpose computing on Graphics Processing Units*). En la práctica, CUDA comprende un modelo de programación basado en *kernels*, *threads*, *blocks* y *grids*, junto con extensiones de C/C++ (y otros lenguajes) y un conjunto de bibliotecas y herramientas de desarrollo.

En esta sección se aborda el modelo de ejecución de CUDA desde la perspectiva de software, el cual sigue el paradigma **SIMT** (*Single Instruction, Multiple Threads*). La unidad fundamental de cómputo es el *kernel*, una función especial que, al ser lanzada, genera un gran número de hilos de ejecución organizados jerárquicamente. Estos hilos se agrupan en bloques (*thread blocks*), y los bloques a su vez conforman una grilla (*grid*), que es asignada a los **SM** (*Streaming Multiprocessors*) del dispositivo.

En la terminología de CUDA, el lanzamiento de un *kernel* crea una instancia de ejecución definida por una configuración de grilla y bloques. Este mecanismo permite exponer paralelismo masivo, reemplazando estructuras secuenciales tradicionales.

Para sustituir los cálculos secuenciales, es necesario un mecanismo que determine qué parte del problema corresponde a cada hilo. CUDA resuelve esta necesidad mediante variables de índice integradas, que permiten a cada hilo identificar su posición dentro de la jerarquía de ejecución. Estas variables cumplen el rol de los índices de bucle en implementaciones secuenciales.

El proceso de lanzamiento de un *kernel*, la construcción de la grilla de ejecución y el esquema de indexación de hilos y bloques se describen en la siguiente sección.

3.1. Kernels en CUDA

El lanzamiento de un *kernel* sigue una sintaxis similar a la de una llamada a función convencional. Sin embargo, CUDA introduce una extensión del lenguaje para especificar la organización paralela de la ejecución. En particular, las dimensiones de la grilla y de los bloques se indican mediante una sintaxis especial con triples ángulos (`<<< >>>`), que se ubica entre el nombre del *kernel* y la lista de argumentos:

```
aKernel<<<Dg, Db>>>(args);
```

Aquí, D_g (número de bloques en la grilla) y D_b (número de hilos por bloque) constituyen la **configuración de ejecución**, que define la estructura del paralelismo.

Para declarar funciones en CUDA, se utilizan calificadores que especifican dónde se ejecuta cada función:

- `__global__`: define *kernels*; son invocados desde el *host* y ejecutados en el *device*.
- `__host__`: funciones que se ejecutan en el *host*.
- `__device__`: funciones que se ejecutan en el *device*.

Los *kernels* no retornan valores, por lo que su tipo de retorno es siempre `void`:

```
__global__ void aKernel(typedArgs)
```

CUDA proporciona además variables integradas que permiten identificar la posición de cada hilo dentro de la jerarquía de ejecución:

- `gridDim`: número de bloques en la grilla.
- `blockDim`: número de hilos por bloque.
- `blockIdx`: índice del bloque dentro de la grilla.
- `threadIdx`: índice del hilo dentro del bloque.

Estas variables permiten mapear de manera directa los datos del problema a los hilos de ejecución, constituyendo la base del modelo de programación en CUDA.

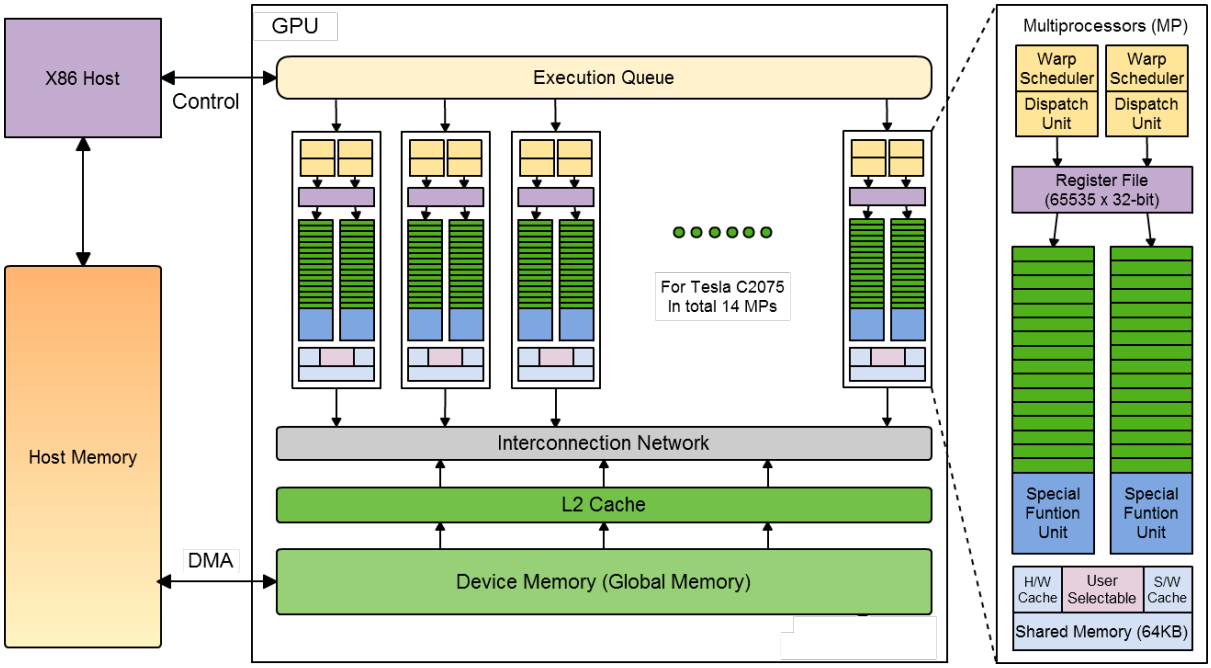


Figura 3: Organización interna de una GPU CUDA.

3.2. Arquitectura de la GPU

La Fig. 3 muestra una arquitectura general de una GPU CUDA y su interacción con el *host* (CPU), ilustrando cómo se organizan los recursos de hardware para ejecutar muchos hilos en paralelo.

A la izquierda se observa el *host* x86 y la memoria del host, desde donde se lanza la ejecución de los kernels. Los datos se transfieren entre la memoria del *host* y la memoria de la GPU mediante DMA (*Direct Memory Access*), lo que permite copiar datos sin involucrar directamente a la CPU durante la transferencia.

En la parte superior de la GPU aparece una cola de ejecución (*Execution Queue*), donde se colocan los kernels que deben ejecutarse en el dispositivo. Estos kernels se distribuyen entre múltiples **multiprocesadores de streaming (SM o MP)**. En el ejemplo indicado en la figura (Tesla C2075), la GPU cuenta con 14 multiprocesadores, cada uno capaz de ejecutar numerosos hilos en paralelo.

Dentro de cada multiprocesador se encuentran varios componentes importantes. Un *warp* scheduler selecciona grupos de hilos listos para ejecutarse y los envía a las unidades de despacho (*dispatch units*), que distribuyen las instrucciones hacia las unidades de cómputo. Cada SM dispone también de un archivo de registros grande, utilizado para almacenar las variables de los hilos activos. Las operaciones especiales (por ejemplo, funciones matemáticas complejas) son realizadas por las unidades de función especial (*Special Function Units*).

Los multiprocesadores están conectados mediante una red de interconexión que les permite acceder a la memoria global del dispositivo (*Device Memory*). Entre los multiprocesadores y la memoria global se encuentra una caché L2, compartida por toda la GPU. Además, cada SM cuenta con memoria compartida (*Shared Memory*), que puede configurarse parcialmente como caché o como memoria controlada por el programador.

En conjunto, esta organización permite que la GPU ejecute miles de hilos simultáneamente, distribuyendo el trabajo entre múltiples multiprocesadores y utilizando diferentes niveles de memoria para mejorar el rendimiento de los accesos a datos.

Cada SM (*Streaming Multiprocessor*, o multiprocesador de flujo) contiene múltiples núcleos de GPU, un pequeño pool de memoria de alta velocidad (SRAM, generalmente compartida entre los núcleos del mismo SM) y varias unidades de ejecución especializadas, como unidades para operaciones de punto flotante, enteras, o para el manejo de texturas y acceso a memoria.

Cada SM opera de manera independiente, manejando múltiples programas (hilos o *threads*) en paralelo. Esto significa que diferentes SMs pueden estar ejecutando distintas partes de un mismo programa, o incluso programas completamente diferentes, lo que permite un alto grado de paralelismo a nivel de chip. La independencia de los SMs es clave para la escalabilidad: al añadir más SMs, la GPU puede procesar más trabajo simultáneamente sin necesidad de rediseñar la arquitectura interna.

3.2.1. Núcleos de la GPU: la unidad de cómputo más pequeña

Un núcleo es la unidad de cómputo más pequeña de un procesador. A diferencia de los núcleos de una CPU, que están optimizados para ejecutar tareas secuenciales complejas con baja latencia y mecanismos avanzados de control (como predicción de saltos o ejecución fuera de orden), los núcleos de una GPU están diseñados para realizar operaciones aritméticas de manera masiva y paralela, especialmente operaciones de punto flotante (FLOP, *Floating Point Operation*).

En las GPU modernas, muchos núcleos simples trabajan en paralelo para ejecutar grandes cantidades de operaciones simultáneamente. Esta organización permite alcanzar un alto número de FLOPS (operaciones de punto flotante por segundo), lo que las hace adecuadas para cargas de trabajo como renderizado gráfico, entrenamiento de modelos de inteligencia artificial, simulaciones científicas y procesamiento de grandes volúmenes de datos.

Además, los núcleos de GPU suelen ser más simples que los de CPU, ya que cuentan con menos lógica de control complejo y menos memoria caché. Sin embargo, al existir cientos o miles de estos núcleos, la GPU puede procesar enormes cantidades de datos en paralelo.

3.2.2. Warps: grupos de hilos en la GPU

En las GPU, los hilos de ejecución se agrupan en unidades llamadas *warps* (en arquitecturas de NVIDIA) o *wavefronts* (en arquitecturas de AMD). Un *warp* está compuesto típicamente por 32 hilos en GPU de NVIDIA, mientras que un *wavefront* contiene 64 hilos en GPU de AMD.

Todos los hilos dentro de un *warp* ejecutan la misma instrucción simultáneamente, pero sobre datos diferentes. Este modelo de ejecución corresponde al paradigma SIMT (*Single Instruction, Multiple Threads*), que permite explotar el paralelismo de datos de manera eficiente.

Si los hilos de un mismo *warp* siguen caminos de ejecución diferentes (por ejemplo, debido a instrucciones condicionales), se produce lo que se conoce como *divergencia de*

warp. En este caso, las diferentes ramas deben ejecutarse de manera secuencial, lo que reduce el paralelismo efectivo y, por lo tanto, el rendimiento.

Por esta razón, al diseñar algoritmos para GPU es conveniente organizar los datos y las cargas de trabajo de manera que se aproveche el tamaño del *warp* (por ejemplo, en múltiplos de 32 en GPU de NVIDIA), evitando así hilos inactivos y mejorando la eficiencia del procesamiento paralelo.

3.3. Hola mundo en CUDA

```
#include <stdio.h>

// Definición de un kernel CUDA.
// __global__ indica que la función se ejecuta en la GPU (device)
// y puede ser invocada desde la CPU (host).
__global__ void kernel(void){}

int main() {
    // Lanzamiento del kernel con la sintaxis <<<Dg, Db>>>:
    // - 1 bloque en la grilla (Dg = 1)
    // - 1 hilo por bloque (Db = 1)
    kernel<<<1,1>>>();
    // Impresión desde la CPU (host)
    printf("Hola mundo!\n");
    return 0;
}
```

Código 1: Ejemplo básico de definición y ejecución de un kernel en CUDA.

El Código 1 muestra la estructura mínima de un programa en CUDA. En primer lugar se define un kernel, que es una función especial marcada con el calificador `__global__`. Este tipo de función es invocada desde el *host* (CPU) pero se ejecuta en el *device* (GPU). En el ejemplo, el kernel no realiza ninguna operación.

En la función `main`, que se ejecuta en la CPU, el kernel se lanza mediante la sintaxis `<<<1,1>>>`. Estos parámetros indican la configuración de ejecución: el primer valor corresponde al número de bloques en la grilla y el segundo al número de hilos por bloque. En este caso se crea una grilla con un bloque que contiene un único hilo.

Finalmente, el programa imprime el mensaje "Hola mundo!" utilizando `printf`, que se ejecuta en el *host*. Este ejemplo se utiliza para ilustrar la sintaxis básica de lanzamiento de kernels en CUDA.

Por otra parte, en el Código 2 mostramos la versión clásica de "Hola Mundo en CUDA" donde el mensaje lo imprime la GPU. A diferencia del ejemplo anterior, donde el kernel no realizaba ninguna operación y el mensaje era impreso por la CPU, en este caso la función ejecutada en la GPU contiene una llamada a printf. De esta manera, el mensaje es generado directamente por el hilo que se ejecuta en el dispositivo. Este ejemplo permite

ilustrar la diferencia entre el código que se ejecuta en el host (CPU) y el que se ejecuta en el device (GPU), así como el mecanismo básico de lanzamiento de kernels en CUDA.

```cuda
#include <stdio.h>

__global__ void kernel() {
    // printf ejecutado desde la GPU.
    // Requiere soporte del runtime CUDA y sincronización posterior
    printf("Hola mundo desde la GPU!\n");
}

int main() {
    // Lanzamiento del kernel con la sintaxis <<<Dg, Db>>>:
    // - 1 bloque en la grilla (Dg = 1)
    // - 1 hilo por bloque (Db = 1)
    kernel<<<1,1>>>();

    // Sincroniza el host con el device:
    // asegura que el kernel termine antes de que el programa finalice
    // (necesario para que el printf en GPU se ejecute correctamente)
    cudaDeviceSynchronize();

    // Fin del programa en el host
    return 0;
}
```

Código 2: Ejemplo básico de definición y ejecución de un kernel en CUDA.

En este último ejemplo se puede modificar la configuración de ejecución del kernel para lanzar varios hilos en paralelo, como se muestra en el Código 3. Cada hilo ejecuta el mismo código del kernel, por lo que el mensaje se imprime varias veces, una por cada hilo creado. Este comportamiento ilustra el modelo de ejecución de CUDA, en el cual múltiples hilos ejecutan simultáneamente la misma función.

El resultado esperado es el siguiente:

```
Hola mundo desde la GPU!
Hola mundo desde la GPU!
Hola mundo desde la GPU!
Hola mundo desde la GPU!
Hola mundo desde la GPU!
```

Cada línea es producida por uno de los cinco hilos que ejecutan el kernel en la GPU. Esto muestra cómo CUDA permite ejecutar el mismo código de manera masivamente paralela.

```cpp
#include <stdio.h>

__global__ void kernel() {
    printf("Hola mundo desde la GPU!\n");
}

int main() {
    / Lanzamiento del kernel con la sintaxis <<<Dg, Db>>>:
    // - Dg = 1 bloque en la grilla
    // - Db = 5 hilos dentro del bloque
    // En total, se lanzan 5 hilos que ejecutan el mismo código en paralelo
    kernel<<<1,5>>>();
    cudaDeviceSynchronize();
    return 0;
}
```

Código 3: Ejemplo básico de definición y ejecución de un kernel en CUDA con mayor cantidad de hilos.

3.4. Otro ejemplo más complejo

Un ejemplo más representativo en CUDA consiste en sumar dos vectores utilizando muchos hilos en paralelo. Cada hilo se encarga de calcular un elemento del vector resultado.

En el Código 4 se muestra cómo utilizar CUDA para realizar la suma de dos vectores en paralelo. El programa define un kernel llamado `sumaVectores`, que se ejecuta en la GPU y calcula la suma de los elementos correspondientes de dos vectores. Cada hilo de ejecución se encarga de procesar un elemento del vector resultado. Para identificar qué elemento debe calcular, cada hilo utiliza su índice dentro del bloque (`threadIdx.x`).

En la función `main`, que se ejecuta en el host (CPU), primero se reservan espacios de memoria en la GPU mediante `cudaMalloc`. Luego se copian los vectores de entrada desde la memoria del host a la memoria del dispositivo utilizando `cudaMemcpy`. A continuación, se lanza el kernel con la configuración `<<<1, N>>>`, lo que crea un bloque con `N` hilos. Cada hilo calcula la suma de un par de elementos de los vectores.

Una vez finalizada la ejecución del kernel, el vector resultado se copia nuevamente desde la memoria del dispositivo a la memoria del host. Finalmente, el programa imprime los resultados y libera la memoria reservada en la GPU. Este ejemplo ilustra el modelo básico de programación en CUDA, donde los datos se transfieren al dispositivo, se ejecuta un cálculo paralelo mediante un kernel, y luego se recuperan los resultados en el host.

4. Compilación de programas CUDA

4.1. Compilación local

Para compilar código CUDA localmente en Linux es necesario cumplir con algunos requisitos.

- Verificar si la GPU puede ejecutar código CUDA. Para ello se puede ejecutar el siguiente comando y luego comprobar si la GPU aparece en la lista de dispositivos compatibles en `https://developer.nvidia.com/cuda-gpus`:

  ```
  $ lspci | grep -i nvidia
  ```

  Si no aparece información sobre la GPU, es posible actualizar la base de datos del hardware PCI con el siguiente comando:

  ```
  $ sudo update-pciids
  ```

4.1.1. Instalación en Ubuntu

Primero se debe instalar el paquete `cuda-keyring`:

```
$ wget https://developer.download.nvidia.com/compute/cuda/repos/<distro>/<arch>/cuda-
$ sudo dpkg -i cuda-keyring_1.1-1_all.deb
```

Luego:

1. Actualizar los repositorios de APT:

   ```
   $ sudo apt update
   ```

2. Instalar el toolkit de CUDA:

   ```
   $ sudo apt install cuda-toolkit
   $ sudo apt install nvidia-gds
   ```

3. Reiniciar el sistema:

   ```
   $ sudo reboot
   ```

En Debian puede ser necesario activar el repositorio `contrib`:

```
$ sudo add-apt-repository contrib
```

4.1.2. Configuración de variables de entorno

La variable `PATH` debe incluir el directorio de binarios de CUDA, y `LD_LIBRARY_PATH` debe contener las bibliotecas:

```
$ export PATH=${PATH}:/usr/local/cuda-13.0/bin
$ export LD_LIBRARY_PATH=${LD_LIBRARY_PATH}:/usr/local/cuda-13.0/lib64
```

Para sistemas de 32 bits:

```
$ export LD_LIBRARY_PATH=${LD_LIBRARY_PATH}:/usr/local/cuda-13.0/lib
```

Finalmente, se puede compilar un ejemplo con el compilador de CUDA:

```
$ nvcc suma.cu
```

4.2. Compilacion Web

Si no se dispone de una GPU con soporte para CUDA, es posible utilizar servicios en línea que permiten compilar y ejecutar código en servidores remotos.

En este caso se puede utilizar el servicio *LeetGPU*, que ofrece un entorno gratuito para escribir, compilar y ejecutar programas CUDA directamente desde el navegador:

`https://leetgpu.com/playground`

5. Bibliografía

- D. Storti and M. Yurtoglu, *CUDA for Engineers: An Introduction to High-Performance Parallel Computing*, 2015.
- NVIDIA Corporation, CUDA C++ Programming Guide, 2025. [Online], disponible en: *https://docs.nvidia.com/cuda/cuda-c-programming-guide/*
- D. B. Kirk and W.-m. W. Hwu, *Programming Massively Parallel Processors: A Hands-on Approach (4th ed.)*, 2023.
- J. Sanders and E. Kandrot, *CUDA by Example: An Introduction to General-Purpose GPU Programming*, 2010.
- L. Scandolo, *Introduccion a OpenCL*, Apunte de la asignatura Arquitectura del Computador LCC-FCEIA, 2024.

```cpp
#include <stdio.h>
__global__ void sumaVectores(int *a, int *b, int *c, int n) {
    int i = threadIdx.x;
    if (i < n) {
        c[i] = a[i] + b[i];
    }
}

int main() {
    const int N = 5;    // Tamaño de los vectores
    // Vectores en memoria del host (CPU)
    int a[N] = {1, 2, 3, 4, 5};
    int b[N] = {10, 20, 30, 40, 50};
    int c[N];

    // Punteros para memoria en el device (GPU)
    int *dev_a, *dev_b, *dev_c;

    // Reserva de memoria en la GPU
    cudaMalloc((void**)&dev_a, N*sizeof(int));
    cudaMalloc((void**)&dev_b, N*sizeof(int));
    cudaMalloc((void**)&dev_c, N*sizeof(int));

    // Copia de datos desde el host (CPU) al device (GPU)
    cudaMemcpy(dev_a, a, N*sizeof(int), cudaMemcpyHostToDevice);
    cudaMemcpy(dev_b, b, N*sizeof(int), cudaMemcpyHostToDevice);

    // Lanzamiento del kernel:
    sumaVectores<<<1,N>>>(dev_a, dev_b, dev_c, N);

    // Copia del resultado desde la GPU al host
    cudaMemcpy(c, dev_c, N*sizeof(int), cudaMemcpyDeviceToHost);

    // Impresión del resultado en el host
    for(int i=0;i<N;i++)
        printf("%d ",c[i]);
    printf("\n");

    // Liberación de memoria en la GPU
    cudaFree(dev_a);
    cudaFree(dev_b);
    cudaFree(dev_c);
    return 0;
}
```

Código 4: Ejemplo de suma vectores utilizando hilos en paralelo.